4 - Architetture software

Cos'è un'architettura software

Una definizione comunemente accettata di architettura software deriva dallo standard IEEE 1471:

> L'organizzazione fondamentale del sistema, incorporata nelle sue componenti, le relazioni tra loro e verso l'ambiente e i principii che governano la sua progettazione e la sua evoluzione.

Si può considerare anche una definizione dello scopo di u'architettura software:

> Il principale intento di un'architettura software è fornire un controllo intellettuale su un sistema sofisticato di enorme complessità.
> ~ Kruchten

Un'architettura software è sempre presente

Ogni sistema ha la sua architettura, che può essere:

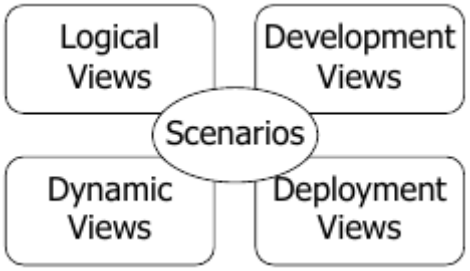
- **intenzionale** (pianificata, progettata) -> forward engineering
  - esplicita
  - documentata
- **non intenzionale** (spontanea, accidentale) -> reverse engineering
  - implicita
  - da ricostruire
  - da documentare

Principi fondamentali

> La separazione di competenze e viste (4+1) spezza la complessità del sistema, consentendone una migliore gestione.
> ~ Kruchten

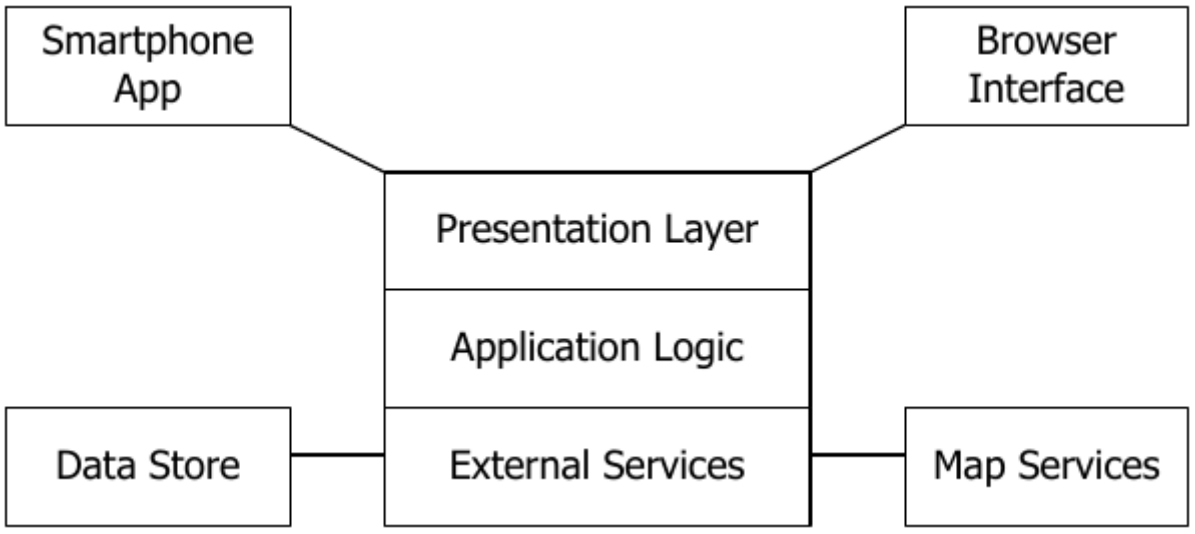
La dicitura "4+1" fa riferimento alle seguenti viste:

- **viste logiche** -> si concentrano sul supporto alle funzionalità e requisiti utente
- **viste di sviluppo** -> si concentrano sui moduli del codice sorgente
- **viste dinamiche** -> si concentrano sugli oggetti e/o i processi a runtime e i flussi di dati
- **viste di deployment** -> si concentrano sulla configurazione, il mapping del software sui server e la distribuzione geografica

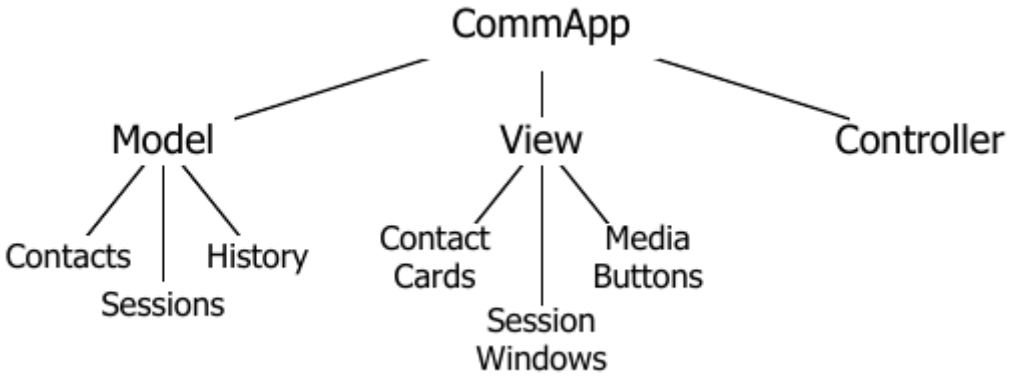


Esempio

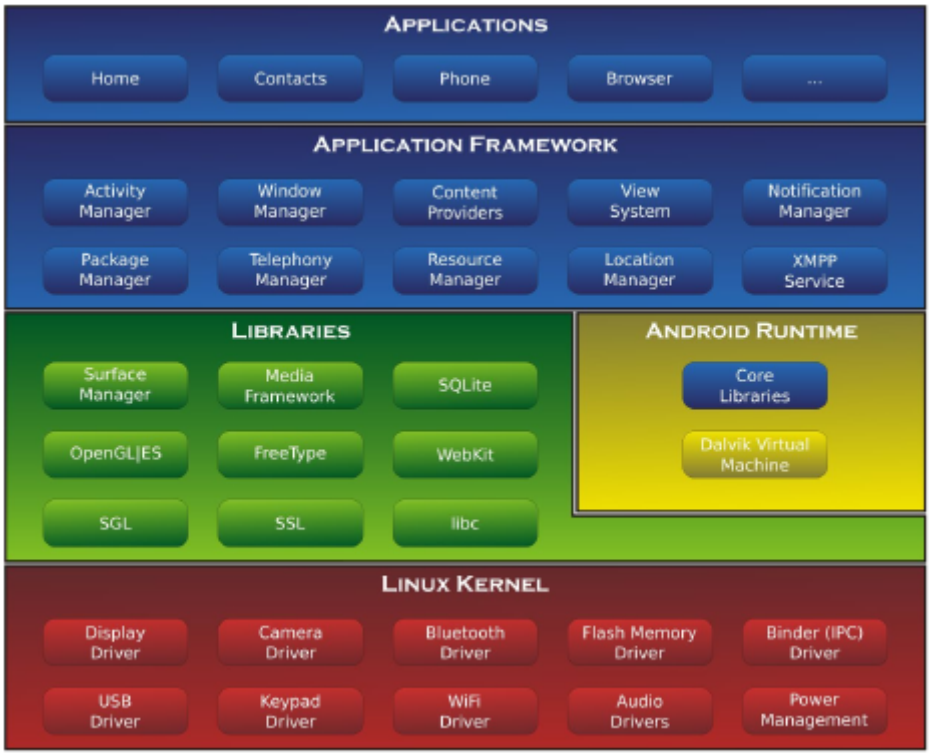
Si consideri un sistema con due interfacce: una per smartphone e una per browser.
Una vista logica del sistema può essere:



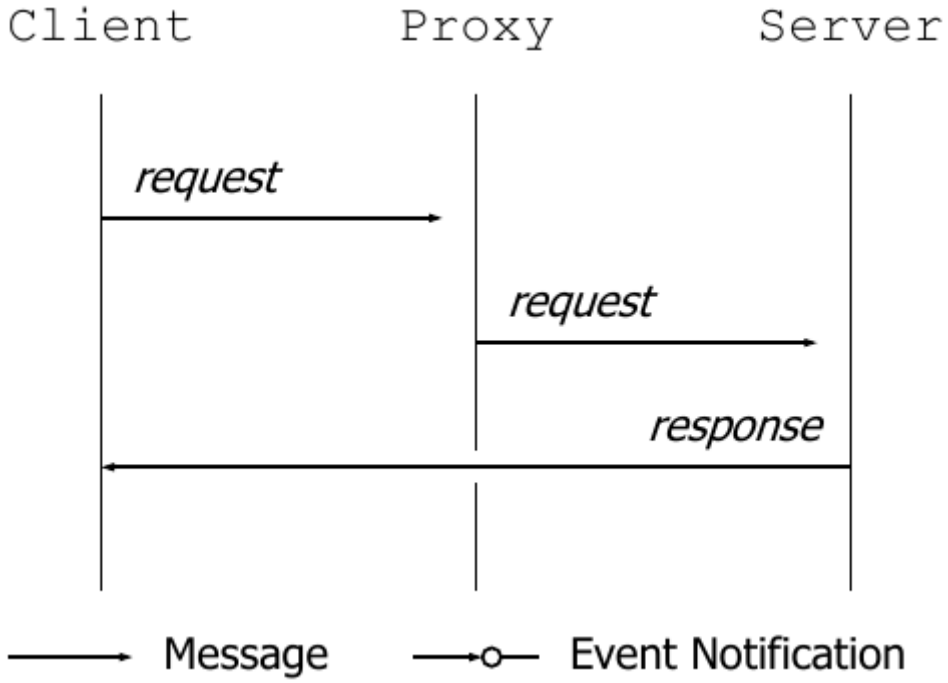
Come vista di sviluppo, si può considerare la gerarchia dei moduli:



o anche lo stack software di Android:



Come vista dinamica invece, si può creare un sequence diagram che mostri le interazioni tra le componenti.



Come trovare un'architettura software

I metodi di progettazione di un'architettura software puntano a derivare l'architettura software dai requisiti funzionali e di qualità.

Ci sono tre principali fattori che guidano la progettazione di un'architettura software: riuso, metodo e intuizione.

- **riuso** -> architettare è un compito difficile che il riuso può aiutare a semplificare. Il fatto che un sistema abbia avuto successo e abbia usato un certo componente, regola o responsabilità favorisce per il riuso di tali soluzioni. Esempi sono gli applicativi bancari e web. La fonte del riuso può essere una versione precedente del sistema, un altro sistema che ne condivide le caratteristiche, pattern architetturali o esperienze a livello di organizzazione. **Domain-specific Software Architecture** e **Product Line Engineering** sono esempi di combinazioni di successo tra architetture software e riuso.
- **metodo** -> ci sono molti linguaggi, modelli di processo e metodi che prescrivono una tecnica sistematica per ridurre le differenze tra architettura software e requisiti.
- **intuizione** -> molti architetti si affidano all'invenzione di elementi di architettura software e le loro relazioni basandosi esclusivamente sulla propria esperienza.

La proporzione tra queste tre fonti di architettura software dipende dall'esperienza di architetti, la loro esperienza formativa e la novità del sistema.

Metodo generale per la progettazione di un'architettura software

Come documentare un'architettura software

Definizioni

Grafo

Si ha un **sistema di interesse**, il quale ha uno o più **stakeholder**. Il sistema ha un'**architettura**, che è descritta da un'appropriata **descrizione**. La descrizione cambia in base a ciò a cui si è interessati: gli stakeholder (*hold* nel senso di avere il potere decisionale) identificano la descrizione.
Gli stakeholder hanno uno o più concern, che consentono di identificare dei punti di vista architetturali. In particolare, il punto di vista è l'insieme di concerti rilevanti per un insieme di stakeholder e che, applicato ad un sistema, dà la vista del sistema. Tra i punti di vista, ne esistono anche alcuni predefiniti. I modelli partecipano a una vista, quindi una vista si compone di almeno un modello. Molto spesso, soprattutto in passato, c'è stata molta attenzione al razionale dell'architettura: perché è fatta in un certo modo? Sarebbe buona pratica, quando si prendono delle decisioni, descrivere i motivi delle decisioni.

Product line engineering

La cilindrata dei motori non cambia al variare della potenza del motore, causa limitazioni della centralina. Ciò è dovuto a un motivo ingegneristico di limitazione dei costi, potendo riutilizzare la stessa centralina per diversi modelli. Anche nell'ingegneria del software, si può applicare il concetto di riutilizzo.

Architettura di riferimento

Nella PLE, non si sviluppa più il codice per un sistema, quanto per un insieme di sistemi. Un'architettura di riferimento è un pattern architetturale predefinito, possibilmente parzialmente o completamente instanziato.

Il riuso sistematico è il riutilizzo predetto delle componenti software, mentre il riuso attivo viene fatto di volta in volta dal caso d'uso.

Tramite una fase di ingegneria della famiglia di prodotti, si sviluppano componenti software per

Scoping

Lo scoping definisce quale funzionalità sono riutilizzabili e quali no. Talvolta, pensare tutti i componenti per il riutilizzo non poterà - prima o poi - ad alcun vantaggio.

Potenziali svantaggi dello sviluppo di componenti riutilizzabili

- **interfaccia complessa**: al fine di
- **rumore dell'organizzazione**: il budget si prende dai prodotti; il family engineering si trova spesso ad affrontare situazioni il team del sistema A deve sviluppare un componente utile anche a B, ma ad un costo maggiore rispetto a quello che sarebbe stato richiesto soltanto da A; in questo caso, occorre trovare un accordo sul budget aggiuntivo.

Stili architetturali

Le applicazioni software, così come le case, hanno il proprio stile. Si hanno le seguenti definizioni:

- **decomposizione in sottosistemi** -> identificazione dei sottosistemi, dei servizi e delle relazioni tra loro
- **stile architetturale** -> pattern per la decomposizione in sottosistemi
- **architettura software** -> istanza di uno stile architetturale
  Nota: un pattern è una soluzione astratta a un problema già esistente.

Applicando uno stile architetturale, si possono identificare i propri sottosistemi. Lo stile architetturale detta la definizione dell'architettura.

Client-server

Uno o più **server** forniscono servizi a istanze di sottosistemi detti **client**. Esiste un'interfaccia dei servizi e spesso i client non conoscono la posizione fisica del server.

Vantaggi: centralizzato, controllo sulle risorse condivise.

Peer-to-peer

È una generalizzazione dell'architettura client-server, in cui i client possono essere server e i server possono essere client.

Esempio: BitTorrent.
Vantaggi: anonimità (seppur parziale), nessun controllo sulle risorse condivise.

Pipes and filters

Un componente legge degli stream di dati come input e produce stream di dati come output.
Si adatta alle applicazioni che richiedono una sequenza di calcoli da eseguire sui dati.
Questo stile architetturale si concentra sulle interazioni dinamiche anziché sulla struttura.

Vantaggi: riuso certificato.

In alcuni contesti, dati i requisiti di alto livello, si procede subito a creare il sequence diagram

Shared data

I dati sono condivisi.